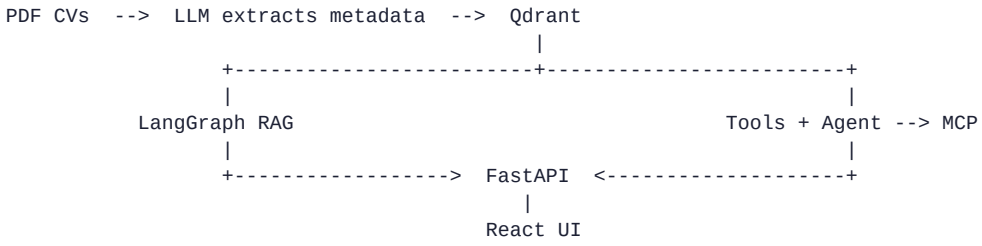
CV Screening Assistant — Teacher briefing

Yassine Nemri · Ihab Lamouchi
https://github.com/Nemri-Yassine/cv-screening-project

One sentence: A recruiter asks in natural language; we search a PDF CV database with RAG + LangGraph, cite the CVs, and show answers in a React app (plus tool calling / MCP).

1. Big picture

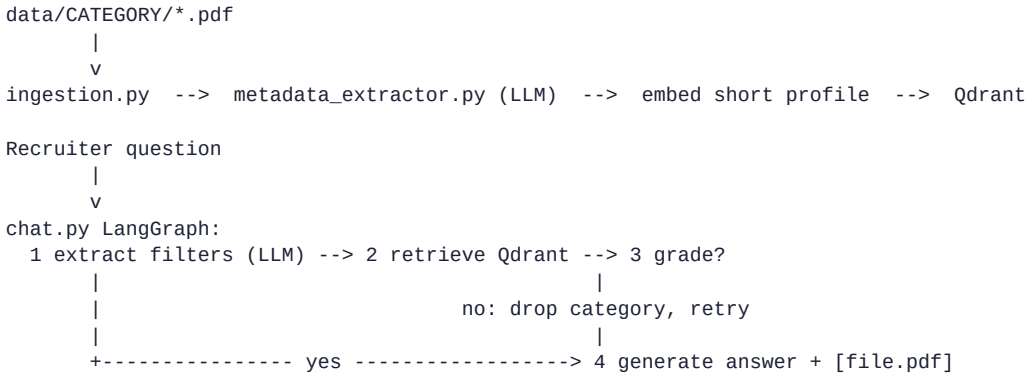


Teacher checklist (mandatory features)

They ask for	We say	Open this file
RAG + metadata + citations	Yes	ingestion.py, metadata_extractor.py, chat.py
LangChain / LangGraph	Yes	chat.py, agent.py
Tool calling / MCP	Yes	tools.py, agent.py, mcp_server.py
Benchmarking	Yes	eval_retrieval.py, eval_generation.py
Web interface	Yes	api.py + frontend/
Fine-tuning (optional)	Bonus done	finetune_slm.ipynb

2. Week 1 — explain THIS first

Week 1 is the foundation: ingest CVs → store in Qdrant → ask questions with a **fixed RAG LangGraph** (not the fancy agent yet).



Say this out loud

1) Each CV PDF is read; an LLM extracts structured metadata (title, years, skills…).
2) We embed a short profile (not the whole messy PDF) → better search.
3) Everything goes into Qdrant with filters (category, years).
4) LangGraph runs: extract filters → search → grade → maybe retry → answer with [filename.pdf] citations.

Where the LLM is called (Week 1)

Step	File	Lines
Create Groq client (metadata)	metadata_extractor.py	18–22
Extract CV fields (structured)	metadata_extractor.py	35–40
Call extraction during ingest	ingestion.py	69

Step	File	Lines
Create Groq client (chat)	chat.py	50–53
Extract search filters	chat.py	96, 147
Grade retrieved CVs	chat.py	97, 182–186
Write final answer	chat.py	98, 168–173
Vector search (no LLM)	chat.py	152–157
Graph wiring (best to show)	chat.py	191–204

Tip: open **chat.py** around line **191** — the whole Week■1 story is there.

Metadata fields

From **metadata_extractor.py** (**CVMetadata**): job_title, years_experience, education_level, skills, summary. Plus folder name = category, and file_name for citations.

3. After Week 1 (only if they ask)

Layer	One line	Open this
Tools	search / read CV / compare / list categories	tools.py (~92–138)
Agent	LLM chooses tools (bind_tools)	agent.py 39–48, 52–60
MCP	same tools for Claude / Cursor	mcp_server.py
API	/api/chat agent or rag mode	api.py 66–87
UI	chat + sources + PDF panel	frontend/src/App.tsx
Retrieval eval	Precision@k, MRR	eval_retrieval.py
Generation eval	LLM-as-judge	eval_generation.py
SLM bonus	LoRA on Qwen 0.5B	finetune_slm.ipynb

4. Honest progress

Done in code	Still to finish
RAG, metadata, citations	Live run: Qdrant + .env + data/ PDFs
LangGraph RAG + agent + MCP	Live demo with real Groq answers
FastAPI + React UI	Technical report PDF + slides
Eval scripts + SLM notebook	Keep daily commits (PHASES.md)
GitHub repo with history	—

Progress line to say: *"We built and published the core RAG pipeline (ingest → Qdrant → LangGraph chat). We added a setup checker after verifying the machine. Next we test the tool-calling agent with real API calls and polish the UI."*

5. Demo commands (when stack is up)

```
docker compose up -d
python scripts/check_setup.py
python ingestion.py
uvicorn api:app --reload --port 8000
cd frontend && npm run dev

Ask: "Find IT profiles with Python and at least 3 years of experience"
```